

RÉALITÉ AUGMENTÉE FROM SCRATCH

Cours Détaillé — Semaines 1 à 6 (Semestre 1 : NkMath)

Modules 01 à 06 — Bibliothèque mathématique from scratch

Règle absolue : Zéro bibliothèque tierce — C++17 + STL uniquement

TEUGUIA TADJUIDJE RODOLF SÉDÉRIS · RIHEN · 3 Semestres · 162h · 21 Modules

SEMAINE 1 — MODULE 01 : IEEE 754 & Arithmétique Flottante

Durée : 6 heures sur 2-3 séances | Semestre 1 — NkMath | C++17 from scratch

Règle absolue de ce cours : aucune bibliothèque tierce. Pas d'OpenCV, pas de GLM, pas d'Eigen, pas de stb_image. Uniquement la STL C++17 et les API système natives. Toute soumission contenant un `#include` d'une bibliothèque interdite est invalidée automatiquement.

1.1 Pourquoi ce module est indispensable

Tout le pipeline de réalité augmentée manipule des nombres flottants : coordonnées 3D, matrices de projection, résidus de calibration, valeurs de pixels. Une erreur d'arrondi invisible dans la résolution d'un système linéaire peut faire diverger Levenberg-Marquardt, produire une homographie incorrecte, ou générer une pose décalée de plusieurs centimètres.

Ce module n'est pas optionnel — c'est la fondation de tout code numérique fiable. La soustraction de deux matrices presque identiques dans la SVD peut perdre 12 bits de précision si on ne comprend pas IEEE 754.

1.2 La représentation binaire des flottants

Structure d'un float 32 bits

Un nombre flottant IEEE 754 de 32 bits (float en C++) est un entier de 32 bits découpé en trois champs :

Structure binaire du float 32 bits

Bit 31 : Signe (0 = positif, 1 = négatif)

Bits 30-23 : Exposant biaisé (8 bits, biais = 127)

Bits 22-0 : Mantisse (23 bits, bit implicite '1.' devant)

Valeur = $(-1)^{\text{signe}} \times 2^{(\text{exposant}-127)} \times 1.\text{mantisse}$

Exemple : 0.1f en binaire

Signe = 0 (positif)

Exposant = 01111011 = 123, donc $2^{(123-127)} = 2^{-4} = 0.0625$

Mantisse = 10011001100110011001101

Valeur = $0.0625 \times 1.60000002384185... = 0.100000001490116...$

=> 0.1 n'est PAS représentable exactement en binaire !

Pour un double 64 bits : signe (1 bit), exposant biaisé (11 bits, biais = 1023), mantisse (52 bits). La précision est environ 15-16 chiffres décimaux significatifs vs 7-8 pour le float.

Accès aux bits d'un flottant en C++17

En C++17, la technique légale pour accéder aux bits d'un flottant sans violer le strict aliasing est `std::memcpy` vers `uint32_t` :

```
#include <cstdint>
#include <cstring>
#include <cstdio>

// Lire les bits d'un float - C++17 légal (pas de type punning UB)
uint32_t floatBits(float f) {
    uint32_t bits;
    std::memcpy(&bits, &f, sizeof(float));
}
```

```

        return bits;
    }

// Afficher les 3 champs d'un float
void inspectFloat(float f) {
    uint32_t bits = floatBits(f);
    uint32_t sign    = (bits >> 31) & 0x1;
    uint32_t exponent = (bits >> 23) & 0xFF;
    uint32_t mantissa = bits          & 0x7FFFFFFF;

    printf("float %.10f\n", f);
    printf("  Bits      : %08X\n", bits);
    printf("  Signe      : %u\n", sign);
    printf("  Exposant: %u (biais=127 → 2^%d)\n", exponent, (int)exponent-127);
    printf("  Mantisse: %06X (1.%06X en hex)\n", mantissa, mantissa);
}

int main() {
    inspectFloat(0.1f);
    inspectFloat(1.0f);
    inspectFloat(0.0f);
    inspectFloat(-0.0f); // -0 != +0 en bits, mais == en valeur !
    return 0;
}

```

1.3 Valeurs spéciales IEEE 754

Le standard IEEE 754 réserve des patterns de bits pour des valeurs spéciales. Il est crucial de les reconnaître car ils apparaissent dans les calculs numériques du pipeline AR :

Valeurs spéciales IEEE 754

+0 : signe=0, exposant=0x00, mantisse=0x000000
 -0 : signe=1, exposant=0x00, mantisse=0x000000 (+0 == -0 mais bits différents)
 +Inf : signe=0, exposant=0xFF, mantisse=0x000000 (ex: 1.0f/0.0f)
 -Inf : signe=1, exposant=0xFF, mantisse=0x000000 (ex: -1.0f/0.0f)
 NaN : signe=X, exposant=0xFF, mantisse != 0 (ex: 0.0f/0.0f, sqrt(-1))
 Subnorm : signe=X, exposant=0x00, mantisse != 0 (très petits nombres)

NaN est contagieux : toute opération avec NaN produit NaN.

NaN != NaN est VRAI — la seule façon de tester NaN est std::isnan(x).

```

#include <cmath>
#include <limits>

// Vérification des valeurs spéciales
float posInf = 1.0f / 0.0f; // +Inf
float negInf = -1.0f / 0.0f; // -Inf
float nanVal = 0.0f / 0.0f; // NaN (quiet NaN)
float nanVal2 = std::sqrt(-1.0f); // NaN

bool isNan = std::isnan(nanVal); // true
bool isInf = std::isinf(posInf); // true
bool isFin = std::isfinite(1.0f); // true

// Piège classique : NaN != NaN !

```

```

bool eq = (nanVal == nanVal);           // FALSE ! Jamais utiliser == pour NaN

// Test correct :
bool isNanCorrect = !(nanVal == nanVal); // true (exploit de la propriété NaN)
bool isNanStd      = std::isnan(nanVal);  // true (méthode recommandée)

```

1.4 L'epsilon machine

L'epsilon machine (ϵ) est la plus petite valeur positive telle que $1.0 + \epsilon \neq 1.0$. C'est la mesure fondamentale de la précision d'un type flottant. Pour float, $\epsilon \approx 1.19\text{e-}7$. Pour double, $\epsilon \approx 2.22\text{e-}16$.

```

// Mesure de l'epsilon machine par boucle (ne PAS utiliser en production)
float measureEpsilon() {
    float eps = 1.0f;
    while(1.0f + eps / 2.0f != 1.0f) {
        eps /= 2.0f;
    }
    return eps;
}

// Valeur standard C++17
float fEps = std::numeric_limits<float>::epsilon(); // 1.19209e-7
double dEps = std::numeric_limits<double>::epsilon(); // 2.22045e-16

// PIÈGE : 1e-7 + 1e-7 != 2e-7 à cause de la précision flottante !
// L'epsilon varie selon la magnitude de x :
// Près de 1.0 : ULP  $\approx 1.19\text{e-}7$ 
// Près de 1000 : ULP  $\approx 1.19\text{e-}4$  (beaucoup moins précis !)

```

1.5 ULP — Unit in the Last Place

L'ULP (Unit in the Last Place) est l'espacement entre deux flottants consécutifs au voisinage d'une valeur donnée. Contrairement à l'epsilon machine (qui est l'ULP près de 1.0), l'ULP varie selon la magnitude de x :

```

// Calculer l'ULP d'un float x
float ulp(float x) {
    // Technique : ajouter 1 au pattern de bits et soustraire x
    uint32_t bits;
    std::memcpy(&bits, &x, 4);
    bits++; // valeur flottante suivante
    float next;
    std::memcpy(&next, &bits, 4);
    return next - x;
}

// Exemples :
// ulp(1.0f)     $\approx 1.19\text{e-}7$  (epsilon machine)
// ulp(100.0f)   $\approx 1.53\text{e-}5$  (moins précis)
// ulp(1000.0f)  $\approx 1.22\text{e-}4$  (encore moins précis)

// Comparaison robuste par ULP :
bool nearlyEqualULP(float a, float b, int maxULP = 4) {
    if(a == b) return true;
    int32_t ia, ib;
    std::memcpy(&ia, &a, 4);
    std::memcpy(&ib, &b, 4);

```

```

// Négatifs : complément à 2 → comparer avec signe
if(ia < 0) ia = 0x80000000 - ia;
if(ib < 0) ib = 0x80000000 - ib;
return std::abs(ia - ib) <= maxULP;
}

```

1.6 Cancellation catastrophique

La cancellation catastrophique est un phénomène qui survient quand on soustrait deux flottants presque égaux : le résultat perd presque tous ses bits significatifs. C'est l'erreur numérique la plus dangereuse du pipeline AR car elle est invisible (pas de NaN, pas d'Inf, juste un résultat faux).

```

// EXEMPLE : Calcul de variance naïve vs Welford

// Méthode naïve - dangereuse !
float varianceNaive(float* data, int n) {
    float sum = 0.0f, sum2 = 0.0f;
    for(int i = 0; i < n; i++) {
        sum += data[i];
        sum2 += data[i] * data[i];
    }
    float mean = sum / n;
    // PROBLÈME : sum2 - sum*sum peut être négatif à cause de l'annulation !
    return (sum2 - sum*sum/n) / (n-1);
}

// Méthode Welford - numériquement stable
float varianceWelford(float* data, int n) {
    float mean = 0.0f, M2 = 0.0f;
    for(int i = 0; i < n; i++) {
        float delta = data[i] - mean;
        mean += delta / (i + 1);
        float delta2 = data[i] - mean;
        M2 += delta * delta2;
    }
    return M2 / (n - 1);
}

// Exemple concret de cancellation :
float a = 1234567.89f;
float b = 1234567.00f;
float diff = a - b; // 0.89 mais stocké comme ~0.875 à cause de l'arrondi !
// Les bits de faible poids de a et b sont déjà erronés avant la soustraction

```

1.7 Sommation de Kahan

L'algorithme de sommation de Kahan compense les erreurs d'arrondi accumulées lors de la somme de nombreux termes. Il maintient une variable de compensation qui capture les bits perdus à chaque addition.

```

// Sommation de Kahan - O(N) comme std::accumulate mais BEAUCOUP plus précis
float kahanSum(const float* data, int n) {
    float sum = 0.0f;
    float comp = 0.0f; // compensation des erreurs perdues

    for(int i = 0; i < n; i++) {
        float y = data[i] - comp; // compenser l'erreur précédente
    }
}

```

```

        float t = sum + y;           // t est grand, y est petit → perte de bits
        comp    = (t - sum) - y;     // capture les bits perdus dans y
        sum     = t;
    }
    return sum;
}

// Démonstration :
// Sommer 1 million de 0.1f
// std::accumulate : résultat = 100011.55 (erreur 11.55 !)
// kahanSum         : résultat = 100000.00 (correct à 6 chiffres)

```

1.8 float vs double — stratégie dans NkMath

La règle adoptée dans ce cours pour NkMath :

Règle float vs double dans NkMath

- Calculs internes (matrices, vecteurs, SVD) : double (64 bits)
Raison : précision nécessaire pour calibration (RMSE < 0.5 pixel)
- Upload GPU (glVertexAttribPointer, glUniformMatrix4fv) : float (32 bits)
Raison : GLSL n'a pas de double natif ; le GPU est optimisé pour float32
- Résultats de calibration (K, distorsion) : double pour stockage
Raison : la calibration accumule des erreurs — le double les minimise
- Types exportés par NkMath : Vec3d (double), Vec3f (float) selon usage

1.9 Comparaison robuste de flottants

```

// NkMath/Float.h — fonctions utilitaires obligatoires
#pragma once
#include <cmath>
#include <limits>
#include <cstdint>
#include <cstring>

namespace NkMath {

// Constantes
constexpr double kEps  = 1e-9;           // epsilon double pour tests
constexpr float  kFEps = 1e-6f;         // epsilon float pour tests

// Valide : ni NaN, ni Inf
inline bool isFiniteValid(double x) { return std::isfinite(x); }
inline bool isFiniteValid(float  x) { return std::isfinite(x); }

// Proche de zéro
inline bool nearlyZero(double x, double eps = kEps) { return std::abs(x) < eps; }
inline bool nearlyZero(float  x, float  eps = kFEps) { return std::abs(x) < eps; }

// Égalité relative
inline bool approxEq(double a, double b, double eps = kEps) {
    if(a == b) return true;

```

```

double maxAB = std::max(std::abs(a), std::abs(b));
return std::abs(a - b) <= eps * std::max(1.0, maxAB);
}

// Somme de Kahan
double kahanSum(const double* data, int n);

} // namespace NkMath

```

1.10 TP — Semaine 1

TP 1 (1h30) — inspectFloat

Implémentez la fonction `inspectFloat(float x)` qui affiche les 3 champs (signe, exposant, mantisse) en binaire sur la console. Testez sur les valeurs suivantes et expliquez chaque résultat :

1. `0.1f` — pourquoi n'est-il pas exact ?
2. `1.0f` — vérifiez que `signe=0`, `exposant=127`, `mantisse=0`
3. `1.0f / 0.0f` — identifier `+Inf`
4. `std::sqrt(-1.0f)` — identifier `NaN`
5. `-0.0f` — comparer les bits avec `+0.0f`
6. `std::numeric_limits<float>::min()` — identifier un subnormal

TP 2 (1h30) — Kahan et Welford

Démontrez empiriquement les problèmes de précision :

7. Créez un tableau de 1 000 000 de copies de `0.1f`
8. Sommez avec `std::accumulate` et avec `kahanSum`. Comparez les résultats à la valeur théorique (100000.0)
9. Calculez la variance de {1e8, 1e8, 1, 2} avec `varianceNaive` et `varianceWelford`. La méthode naïve peut donner une variance négative !
10. Mesurez `epsilon machine` par boucle et comparez à `std::numeric_limits<float>::epsilon()`

TP 3 (1h30) — NkMath/Float.h

Créez le fichier `NkMath/Float.h` avec toutes les fonctions utilitaires décrites dans la section 1.9. Écrivez 30 tests unitaires minimum couvrant :

11. `isFiniteValid` sur `NaN`, `+Inf`, `-Inf`, `0.0`, `1.0`
12. `nearlyZero` avec différentes valeurs `epsilon`
13. `approxEq` sur des paires (a, b) avec différentes tolérances
14. `kahanSum` vs `accumulate` sur 10 cas

SEMAINE 2 — MODULE 02 : Vecteurs — Vec2d, Vec3d, Vec4d

Durée : 9 heures sur 3 séances | Semestre 1 — NkMath | C++17 from scratch

2.1 Pourquoi les vecteurs sont l'alphabet du pipeline 3D

Les vecteurs sont la structure de données la plus utilisée du pipeline AR. Chaque position dans l'espace est un Vec3d. Chaque direction (normale, axe de rotation, rayon de lumière) est un Vec3d normalisé. Chaque pixel de l'image a une position Vec2d. Chaque coordonnée homogène transmise à la matrice de projection est un Vec4d.

Le test d'appartenance d'un point à un triangle (rasterisation) utilise des dot products. La normale d'une face est le cross product de deux arêtes. Un tableau de Vec3d est envoyé directement à glVertexAttribPointer sans copie si le layout mémoire est correct. Sans une implémentation rigoureuse avec static_assert sur sizeof, rien d'autre dans NkMath ne fonctionne.

2.2 Implémentation de Vec2d

```
#pragma once
#include "Float.h"
#include <cmath>
#include <cassert>
#include <cstdio>

namespace NkMath {

struct Vec2d {
    double x, y;

    // Constructeurs
    Vec2d() : x(0.0), y(0.0) {}
    Vec2d(double x, double y) : x(x), y(y) {}
    explicit Vec2d(double s) : x(s), y(s) {} // fill constructor

    // Accès par index — & pour permettre l'écriture
    double& operator[](int i) {
        assert(i >= 0 && i < 2 && "Vec2d index out of bounds");
        return (&x)[i]; // garanti contigu en C++17
    }
    const double& operator[](int i) const {
        assert(i >= 0 && i < 2 && "Vec2d index out of bounds");
        return (&x)[i];
    }

    // Opérateurs arithmétiques
    Vec2d operator+(const Vec2d& o) const { return {x+o.x, y+o.y}; }
    Vec2d operator-(const Vec2d& o) const { return {x-o.x, y-o.y}; }
    Vec2d operator*(double s) const { return {x*s, y*s}; }
    Vec2d operator/(double s) const { assert(!nearlyZero(s)); return {x/s, y/s}; }
    Vec2d operator-() const { return {-x, -y}; }

    // Opérateurs composés
    Vec2d& operator+=(const Vec2d& o) { x+=o.x; y+=o.y; return *this; }
    Vec2d& operator-=(const Vec2d& o) { x-=o.x; y-=o.y; return *this; }
    Vec2d& operator*=(double s) { x*=s; y*=s; return *this; }
```



```

// Norme et normalisation
double Norm2() const { return x*x + y*y; }
double Norm() const { return std::sqrt(Norm2()); }

Vec2d Normalized() const {
    double n = Norm();
    if(nearlyZero(n)) return Vec2d(0.0); // vecteur nul → retourner zéro
    return {x/n, y/n};
}

bool IsNormalized(double eps = kEps) const {
    return approxEq(Norm2(), 1.0, eps);
}

void Print() const { printf("Vec2d(%.6f, %.6f)\n", x, y); }
};

// Dot product : a·b = |a||b|cos(θ)
inline double Dot(const Vec2d& a, const Vec2d& b) {
    return a.x*b.x + a.y*b.y;
}

// Cross 2D (scalaire) : composante z du produit vectoriel 3D
// Positif si b est à gauche de a, négatif si à droite
inline double Cross2D(const Vec2d& a, const Vec2d& b) {
    return a.x*b.y - a.y*b.x;
}

// Lerp linéaire
inline Vec2d Lerp(const Vec2d& a, const Vec2d& b, double t) {
    return {a.x + (b.x-a.x)*t, a.y + (b.y-a.y)*t};
}

// Opérateur externe scalaire * Vec2d
inline Vec2d operator*(double s, const Vec2d& v) { return v * s; }

// Garantie de layout mémoire pour glVertexAttribPointer
static_assert(sizeof(Vec2d) == 16, "Vec2d must be 16 bytes (2 doubles)");
static_assert(offsetof(Vec2d, x) == 0, "x must be first");
static_assert(offsetof(Vec2d, y) == 8, "y must be at offset 8");

} // namespace NkMath

```

2.3 Le dot product — interprétation géométrique

Le dot product (produit scalaire) est l'opération vectorielle la plus utilisée du pipeline. Formule : $a \cdot b = a.x*b.x + a.y*b.y + a.z*b.z = |a||b|\cos(\theta)$.

Interprétation géométrique du dot product

- $a \cdot b = 0$ → a et b sont perpendiculaires ($\theta = 90^\circ$)
- $a \cdot b = 1$ → a et b sont dans la même direction (si normalisés, $\theta = 0^\circ$)
- $a \cdot b = -1$ → a et b sont opposés (si normalisés, $\theta = 180^\circ$)
- $a \cdot b > 0$ → angle aigu ($< 90^\circ$)
- $a \cdot b < 0$ → angle obtus ($> 90^\circ$)

Applications dans le pipeline AR :

- Test face avant/arrière : $\text{dot}(\text{normal}, \text{cameraDir}) > 0 \rightarrow \text{face arri\`ere}$
- Illumination Phong : $\text{dot}(\text{normal}, \text{lightDir}) \rightarrow \text{intensit\`e diffuse}$
- Projection d'un point sur une droite
- Test d'appartenance à un plan

2.4 Le cross product 3D — implémentation et interprétation

```
// Vec3d avec cross product
struct Vec3d {
    double x, y, z;

    // ... (même structure que Vec2d pour les opérateurs de base)

    double Norm2() const { return x*x + y*y + z*z; }
    double Norm()  const { return std::sqrt(Norm2()); }
    Vec3d Normalized() const {
        double n = Norm();
        if(nearlyZero(n)) return Vec3d(0.0);
        return {x/n, y/n, z/n};
    }
};

// Cross product 3D : a × b
// Résultat : vecteur perpendiculaire à a et b
// Norme = |a||b|sin(θ) = aire du parallélogramme formé par a et b
// Direction : règle de la main droite (CCW positif en OpenGL)
inline Vec3d Cross(const Vec3d& a, const Vec3d& b) {
    return {
        a.y*b.z - a.z*b.y, // composante x
        a.z*b.x - a.x*b.z, // composante y
        a.x*b.y - a.y*b.x  // composante z
    };
}

// Vérification :
// Cross({1,0,0}, {0,1,0}) = {0,0,1} (axe Z = X × Y)
// Cross({0,1,0}, {1,0,0}) = {0,0,-1} (non-commutatif !)
// Cross({1,0,0}, {1,0,0}) = {0,0,0} (vecteurs parallèles → zéro)
```

2.5 Projection et rejection

```
// Projection de a sur b : composante de a dans la direction de b
// proj(a,b) = b × (a·b / |b|²)
inline Vec3d Project(const Vec3d& a, const Vec3d& b) {
    double b2 = b.Norm2();
    assert(!nearlyZero(b2) && "Cannot project onto zero vector");
    return b * (Dot(a, b) / b2);
}

// Rejection de a par rapport à b : composante de a perpendiculaire à b
// reject(a,b) = a - proj(a,b)
inline Vec3d Reject(const Vec3d& a, const Vec3d& b) {
    return a - Project(a, b);
}
```

```

}

// Application : décomposer le vecteur vitesse en composantes normale et
// tangentielle
// velocity_n = Project(velocity, normal)    ← vitesse vers la surface
// velocity_t = Reject(velocity, normal)    ← vitesse le long de la surface
// Utilisé dans la résolution de collision : n'inverser que velocity_n

```

2.6 Gram-Schmidt — orthogonalisation d'une base

```

// Orthogonalisation de Gram-Schmidt
// Entrée : 3 vecteurs (peuvent être non orthogonaux)
// Sortie : 3 vecteurs orthonormaux (base orthonormale)
struct OrthoBasis {
    Vec3d u, v, w; // garantis orthogonaux et normalisés
};

inline OrthoBasis GramSchmidt(Vec3d a, Vec3d b, Vec3d c) {
    // Étape 1 : normaliser a → u
    Vec3d u = a.Normalized();
    assert(!nearlyZero(u.Norm()) && "First vector is zero");

    // Étape 2 : ôter la composante u de b → v
    Vec3d v = (b - Project(b, u)).Normalized();
    assert(!nearlyZero(v.Norm()) && "Vectors are collinear");

    // Étape 3 : ôter les composantes u et v de c → w
    Vec3d w = (c - Project(c, u) - Project(c, v)).Normalized();
    assert(!nearlyZero(w.Norm()) && "Vectors are coplanar");

    return {u, v, w};
}

// Utilisation : constructeur LookAt (semaine 3), décomposition polaire (M05)

```

2.7 Coordonnées homogènes — Vec4d

Les coordonnées homogènes permettent de représenter points et directions de façon unifiée, et d'encoder translations, rotations et projections dans une seule multiplication matricielle 4×4.

```

struct Vec4d {
    double x, y, z, w;

    Vec4d() : x(0), y(0), z(0), w(0) {}
    Vec4d(double x, double y, double z, double w) : x(x), y(y), z(z), w(w) {}
    Vec4d(const Vec3d& v, double w) : x(v.x), y(v.y), z(v.z), w(w) {}

    // Conversion vers Vec3d — déhomogénéisation
    Vec3d ToVec3() const {
        assert(!nearlyZero(w) && "Cannot dehomogenize: w is zero (direction)");
        return {x/w, y/w, z/w};
    }

    // Accès
    double& operator[](int i) { assert(i<4); return (&x)[i]; }
};

```

```

// Convention :
// w = 1.0 → POINT dans l'espace (la translation de Mat4 s'applique)
// w = 0.0 → DIRECTION (la translation ne s'applique PAS)

// Exemple :
Vec4d point      = {1.0, 2.0, 3.0, 1.0}; // point (1,2,3)
Vec4d direction  = {0.0, 1.0, 0.0, 0.0}; // direction Y+ (vers le haut)

// Projection (semaine 3) : Mat4d × Vec4d → Vec4d → ToVec3() (÷ par w)

// Garantie layout mémoire
static_assert(sizeof(Vec4d) == 32, "Vec4d must be 32 bytes");

```

2.8 TP — Semaine 2

TP 1 (1h30) — Vec2d complet

Implémentez Vec2d avec tous les opérateurs. Écrivez 20 tests unitaires avec assert couvrant :

15. Dot product : vérifiez que $(1,0) \cdot (0,1) = 0$, $(1,0) \cdot (1,0) = 1$, $(3,4) \cdot (3,4) = 25$
16. Cross2D : vérifiez $(1,0) \times (0,1) = 1$, $(0,1) \times (1,0) = -1$
17. Normalisation : vérifiez que $\|(3,4).Normalized()\| = 1.0$ à kEps près
18. Opérateur [] : vérifiez l'accès et la modification
19. `static_assert(sizeof(Vec2d)==16)`

TP 2 (1h30) — Vec3d avec Gram-Schmidt

Implémentez Vec3d avec cross product 3D. Tests :

20. `Cross({1,0,0},{0,1,0}) = {0,0,1}` — règle main droite
21. `Cross({0,1,0},{1,0,0}) = {0,0,-1}` — non-commutatif
22. Gram-Schmidt sur 10 triplets aléatoires — vérifiez l'orthonormalité ($|u|=1$, $u \cdot v=0$, $u \cdot w=0$, $v \cdot w=0$)
23. Project et Reject : vérifiez que `Project(a,b) + Reject(a,b) == a`

TP 3 (1h30) — Vec4d et projection perspective simple

Implémentez Vec4d avec déhomogénéisation. Projetez les 8 coins d'un cube unitaire $[-0.5,0.5]^3$ en 2D :

24. Distance caméra-cube : `z_cam = 2.0`
25. Projection simple : $u = f_x * (X/Z) + c_x$, $v = f_y * (Y/Z) + c_y$ avec $f_x=f_y=500$, $c_x=c_y=256$
26. Créez une image NkImage 512×512 (approximation PPM) et affichez les 8 coins projetés comme des points
27. Tracez les 12 arêtes du cube dans l'image

SEMAINE 3 — MODULE 03 : Matrices — Mat3d, Mat4d

Durée : 9 heures sur 3 séances | Semestre 1 — NkMath | C++17 from scratch

3.1 Convention column-major — critique pour OpenGL

En mathématiques, on note habituellement les matrices en row-major (ligne par ligne). OpenGL utilise la convention column-major (colonne par colonne). Cette différence est fondamentale : une matrice transposée par erreur dans l'upload GPU produit des transformations incorrectes.

Convention column-major OpenGL

Matrice 4×4 mathématique (row-major) :

```
| m00 m01 m02 m03 |  
| m10 m11 m12 m13 |  
| m20 m21 m22 m23 |  
| m30 m31 m32 m33 |
```

Stockage column-major en mémoire (OpenGL) :

```
data[0]=m00, data[1]=m10, data[2]=m20, data[3]=m30, // colonne 0  
data[4]=m01, data[5]=m11, data[6]=m21, data[7]=m31, // colonne 1  
...
```

Accès en C++ : `data[col*4 + row] = m(row, col)`

`glUniformMatrix4fv(loc, 1, GL_FALSE, data)` — `GL_FALSE` = déjà column-major

`glUniformMatrix4fv(loc, 1, GL_TRUE, data)` — `GL_TRUE` = transposer (si row-major)

```
#pragma once  
#include "Vec4d.h"  
#include <array>  
  
namespace NkMath {  
  
struct Mat4d {  
    // Stockage column-major : data[col*4 + row]  
    double data[16];  
  
    Mat4d() { std::fill(data, data+16, 0.0); }  
  
    // Accès row,col  
    double& operator()(int row, int col) {  
        assert(row>=0 && row<4 && col>=0 && col<4);  
        return data[col*4 + row]; // column-major !  
    }  
    const double& operator()(int row, int col) const {  
        return data[col*4 + row];  
    }  
  
    // Matrice identité  
    static Mat4d Identity() {  
        Mat4d m;  
        m(0,0) = m(1,1) = m(2,2) = m(3,3) = 1.0;  
        return m;  
    }  
}
```

```

// Produit matriciel O(64 multiplications pour 4x4)
Mat4d operator*(const Mat4d& o) const {
    Mat4d result;
    for(int row = 0; row < 4; row++)
        for(int col = 0; col < 4; col++) {
            double sum = 0.0;
            for(int k = 0; k < 4; k++)
                sum += (*this)(row, k) * o(k, col);
            result(row, col) = sum;
        }
    return result;
}

// Multiplication Mat4d x Vec4d
Vec4d operator*(const Vec4d& v) const {
    return {
        (*this)(0,0)*v.x + (*this)(0,1)*v.y + (*this)(0,2)*v.z +
        (*this)(0,3)*v.w,
        (*this)(1,0)*v.x + (*this)(1,1)*v.y + (*this)(1,2)*v.z +
        (*this)(1,3)*v.w,
        (*this)(2,0)*v.x + (*this)(2,1)*v.y + (*this)(2,2)*v.z +
        (*this)(2,3)*v.w,
        (*this)(3,0)*v.x + (*this)(3,1)*v.y + (*this)(3,2)*v.z +
        (*this)(3,3)*v.w
    };
}

// Transposée
Mat4d Transposed() const {
    Mat4d t;
    for(int r = 0; r < 4; r++)
        for(int c = 0; c < 4; c++)
            t(r,c) = (*this)(c,r);
    return t;
}

// Pointer vers les données pour upload GPU
const double* DataPtr() const { return data; }
// Pour upload float GPU (convertir en float):
void ToFloat(float out[16]) const {
    for(int i=0; i<16; i++) out[i] = (float)data[i];
}
};

} // namespace NkMath

```

3.2 Inverse par Gauss-Jordan avec pivot partiel

```

// Retourne false si la matrice est singulière (non inversible)
bool Inverse(const Mat4d& m, Mat4d& out) {
    // Matrice augmentée [m | I]
    double aug[4][8];
    for(int r = 0; r < 4; r++)
        for(int c = 0; c < 4; c++) {
            aug[r][c] = m(r,c);
            aug[r][c+4] = (r == c) ? 1.0 : 0.0;
        }
}

```

```

for(int col = 0; col < 4; col++) {
    // Pivot partiel : trouver la ligne avec le max absolu dans cette colonne
    int pivotRow = col;
    for(int r = col+1; r < 4; r++)
        if(std::abs(aug[r][col]) > std::abs(aug[pivotRow][col]))
            pivotRow = r;

    // Swap des lignes col et pivotRow
    if(pivotRow != col)
        for(int c = 0; c < 8; c++)
            std::swap(aug[col][c], aug[pivotRow][c]);

    // Vérification singularité
    if(nearlyZero(aug[col][col])) return false;

    // Normaliser la ligne pivot
    double inv = 1.0 / aug[col][col];
    for(int c = 0; c < 8; c++) aug[col][c] *= inv;

    // Élimination sur les autres lignes
    for(int r = 0; r < 4; r++) {
        if(r == col) continue;
        double factor = aug[r][col];
        for(int c = 0; c < 8; c++)
            aug[r][c] -= factor * aug[col][c];
    }
}

// Extraire la partie droite [I | A-1]
for(int r = 0; r < 4; r++)
    for(int c = 0; c < 4; c++)
        out(r,c) = aug[r][c+4];
return true;
}

```

3.3 Matrice de rotation — formule de Rodrigues

La rotation d'angle θ autour d'un axe unitaire $\mathbf{n} = (n_x, n_y, n_z)$ est donnée par la formule de Rodrigues :

```

Mat4d RotateAxis(const Vec3d& axis, double angleRad) {
    Vec3d n = axis.Normalized();
    double c = std::cos(angleRad);
    double s = std::sin(angleRad);
    double t = 1.0 - c;

    Mat4d R = Mat4d::Identity();
    // Formule de Rodrigues
    R(0,0) = t*n.x*n.x + c;
    R(0,1) = t*n.x*n.y - s*n.z;
    R(0,2) = t*n.x*n.z + s*n.y;

    R(1,0) = t*n.x*n.y + s*n.z;
    R(1,1) = t*n.y*n.y + c;
    R(1,2) = t*n.y*n.z - s*n.x;

```

```

    R(2,0) = t*n.x*n.z - s*n.y;
    R(2,1) = t*n.y*n.z + s*n.x;
    R(2,2) = t*n.z*n.z + c;

    return R;
}

// Vérification :
// RotateAxis({0,1,0}, PI/2) × {1,0,0,1} = {0,0,-1,1} (rotation de X vers -Z)

```

3.4 LookAt — matrice View

```

// Construire la matrice View (caméra regarde target depuis eye)
Mat4d LookAt(const Vec3d& eye, const Vec3d& target, const Vec3d& worldUp) {
    // Axe forward = direction regardée = normalize(target - eye)
    Vec3d f = (target - eye).Normalized();

    // Axe right = perpendiculaire à forward et worldUp
    Vec3d r = Cross(f, worldUp).Normalized();

    // Axe up corrigé = perpendiculaire à right et forward
    Vec3d u = Cross(r, f);

    // View = R * T (rotater puis translater)
    Mat4d V = Mat4d::Identity();
    V(0,0) = r.x; V(0,1) = r.y; V(0,2) = r.z;
    V(1,0) = u.x; V(1,1) = u.y; V(1,2) = u.z;
    V(2,0) = -f.x; V(2,1) = -f.y; V(2,2) = -f.z;
    // Translation : t = -R × eye
    V(0,3) = -Dot(r, eye);
    V(1,3) = -Dot(u, eye);
    V(2,3) = Dot(f, eye);
    return V;
}

// Construire Mat4d depuis [R3×3 | t3×1] de solvePnP (Semestre 3)
Mat4d FromRT(const Mat3d& R, const Vec3d& t) {
    Mat4d M = Mat4d::Identity();
    for(int r=0; r<3; r++)
        for(int c=0; c<3; c++)
            M(r,c) = R(r,c);
    M(0,3) = t.x; M(1,3) = t.y; M(2,3) = t.z;
    return M;
}

```

3.5 TRS — Translation, Rotation, Scale

```

Mat4d Translate(const Vec3d& t) {
    Mat4d M = Mat4d::Identity();
    M(0,3) = t.x; M(1,3) = t.y; M(2,3) = t.z;
    return M;
}

Mat4d Scale(const Vec3d& s) {
    Mat4d M = Mat4d::Identity();
    M(0,0) = s.x; M(1,1) = s.y; M(2,2) = s.z;
}

```



```

    return M;
}

// TRS = T × R × S — L'ORDRE EST CRUCIAL !
// 1. Mettre à l'échelle l'objet (autour de son origine locale)
// 2. Tourner l'objet mis à l'échelle
// 3. Translater vers la position monde
Mat4d TRS(const Vec3d& t, const Vec3d& axis, double angle, const Vec3d& s) {
    return Translate(t) * RotateAxis(axis, angle) * Scale(s);
}

```

3.6 TP — Semaine 3

TP 1 (1h30) — Mat4d et inverse

Implémentez Mat4d avec produit et inverse. Tests :

28. $M \times \text{Identity}() == M$ pour 10 matrices aléatoires
29. $M \times M^{-1} == \text{Identity}()$ à $1e-10$ près pour 10 matrices non-singulières
30. Inverse d'une matrice singulière doit retourner false
31. $\text{RotateAxis}(\{0,1,0\}, \text{PI}/2) \times \{1,0,0,1\} == \{0,0,-1,1\}$

TP 2 (1h30) — Rasteriseur logiciel avec LookAt

Implémentez un rasteriseur logiciel simple dans une NkImage 512×512 :

32. Placez un cube unitaire centré à l'origine
33. Créez une vue avec LookAt(eye={0,1,3}, target={0,0,0}, up={0,1,0})
34. Projection perspective simple avec fov=60°
35. Affichez les 12 arêtes du cube en rouge dans l'image
36. Sauvegardez en format PPM (P6 binaire)
37. Faites tourner le cube d'un angle différent à chaque frame (10 frames)

TP 3 (1h30) — TRS et décomposition

Implémentez TRS complet et vérifiez la décomposition inverse :

38. Pour 20 triplets (T, R, S) aléatoires, construisez $M = \text{TRS}(T, R, S)$
39. Décomposez M pour retrouver T, R et S
40. Vérifiez que les valeurs retrouvées correspondent aux valeurs originales

SEMAINE 4 — MODULE 04 : Quaternions

Durée : 6 heures sur 2 séances | Semestre 1 — NkMath | C++17 from scratch

4.1 Le problème du gimbal lock

Avant de comprendre pourquoi les quaternions existent, il faut comprendre le problème qu'ils résolvent : le gimbal lock des angles d'Euler.

Les angles d'Euler représentent une rotation par trois angles successifs (ex: yaw, pitch, roll). Le problème survient quand $\text{pitch} = \pm 90^\circ$: les axes yaw et roll deviennent coplanaires et le système perd un degré de liberté. La rotation dans un sens particulier ne peut plus être encodée.

En pratique dans le pipeline AR : si vous interpoliez entre deux orientations avec des angles d'Euler via LERP, le chemin d'interpolation n'est pas le chemin le plus court sur la sphère des rotations. Cela produit des mouvements non-naturels et visuellement incorrects.

4.2 Définition des quaternions

Un quaternion $q = w + xi + yj + zk$ est un nombre hypercomplexe avec $i^2 = j^2 = k^2 = ijk = -1$. Un quaternion unitaire ($\|q\| = 1$) représente une rotation de $2 \cdot \arccos(w)$ radians autour de l'axe (x, y, z) .

```
#pragma once
#include "Vec3d.h"
#include "Mat3d.h"
#include <cmath>

namespace NkMath {

struct Quat {
    double w, x, y, z; // w = partie réelle, (x,y,z) = partie imaginaire

    Quat() : w(1), x(0), y(0), z(0) {} // identité = pas de rotation
    Quat(double w, double x, double y, double z) : w(w), x(x), y(y), z(z) {}

    double Norm2() const { return w*w + x*x + y*y + z*z; }
    double Norm() const { return std::sqrt(Norm2()); }

    Quat Normalized() const {
        double n = Norm();
        assert(!nearlyZero(n));
        return {w/n, x/n, y/n, z/n};
    }

    // Conjugué : inverse la rotation
    Quat Conjugate() const { return {w, -x, -y, -z}; }

    // Inverse : pour quaternion unitaire, Inverse() == Conjugate()
    Quat Inverse() const {
        double n2 = Norm2();
        assert(!nearlyZero(n2));
        return {w/n2, -x/n2, -y/n2, -z/n2};
    }

    // Produit de Hamilton (composition de rotations)
    Quat operator*(const Quat& o) const {
        return {
            w*o.w - x*o.x - y*o.y - z*o.z,
```

```

        w*o.x + x*o.w + y*o.z - z*o.y,
        w*o.y - x*o.z + y*o.w + z*o.x,
        w*o.z + x*o.y - y*o.x + z*o.w
    };
}
};

// Quaternion depuis axe-angle (formule de Rodrigues)
inline Quat FromAxisAngle(const Vec3d& axis, double angleRad) {
    Vec3d n = axis.Normalized();
    double s = std::sin(angleRad / 2.0);
    double c = std::cos(angleRad / 2.0);
    return {c, n.x*s, n.y*s, n.z*s};
}

// Rotation d'un vecteur par un quaternion
//  $v' = q \otimes (0, v) \otimes q^*$  (version optimisée sans construire 2 quaternions)
inline Vec3d Rotate(const Quat& q, const Vec3d& v) {
    Vec3d qVec = {q.x, q.y, q.z};
    Vec3d uv = Cross(qVec, v);
    Vec3d uuv = Cross(qVec, uv);
    return v + (uv * (2.0 * q.w)) + (uuv * 2.0);
}

} // namespace NkMath

```

4.3 SLERP — interpolation sphérique

```

// SLERP : Spherical Linear interPolation
// Suit le chemin le plus court sur la sphère des quaternions unitaires
inline Quat Slerp(Quat a, Quat b, double t) {
    // Calculer le cosinus de l'angle entre a et b
    double cosAngle = a.w*b.w + a.x*b.x + a.y*b.y + a.z*b.z;

    // Si cos < 0, les quaternions sont sur des hémisphères opposés
    // → inverser b pour prendre le chemin le plus court
    if(cosAngle < 0.0) {
        b.w = -b.w; b.x = -b.x; b.y = -b.y; b.z = -b.z;
        cosAngle = -cosAngle;
    }

    double k0, k1;
    if(cosAngle > 0.9999) {
        // Angle trop petit → LERP pour éviter division par zéro
        k0 = 1.0 - t;
        k1 = t;
    } else {
        // SLERP standard
        double angle = std::acos(cosAngle);
        double sinAngle = std::sin(angle);
        k0 = std::sin((1.0-t) * angle) / sinAngle;
        k1 = std::sin(t * angle) / sinAngle;
    }

    return Quat {
        k0*a.w + k1*b.w,
        k0*a.x + k1*b.x,

```

```

        k0*a.y + k1*b.y,
        k0*a.z + k1*b.z
    }.Normalized();
}

```

4.4 Méthode de Shepperd — Quat ↔ Mat3

```

// Quat vers Mat3 — pour upload dans le shader (via Mat4)
Mat3d ToMat3(const Quat& q) {
    double xx=q.x*q.x, yy=q.y*q.y, zz=q.z*q.z;
    double xy=q.x*q.y, xz=q.x*q.z, yz=q.y*q.z;
    double wx=q.w*q.x, wy=q.w*q.y, wz=q.w*q.z;
    Mat3d R;
    R(0,0)=1-2*(yy+zz); R(0,1)=2*(xy-wz); R(0,2)=2*(xz+wy);
    R(1,0)=2*(xy+wz); R(1,1)=1-2*(xx+zz); R(1,2)=2*(yz-wx);
    R(2,0)=2*(xz-wy); R(2,1)=2*(yz+wx); R(2,2)=1-2*(xx+yy);
    return R;
}

// Mat3 vers Quat — Méthode de Shepperd (stable numériquement)
Quat FromMat3(const Mat3d& R) {
    double trace = R(0,0) + R(1,1) + R(2,2);
    Quat q;
    if(trace > 0) {
        double s = 0.5 / std::sqrt(trace + 1.0);
        q.w = 0.25 / s;
        q.x = (R(2,1) - R(1,2)) * s;
        q.y = (R(0,2) - R(2,0)) * s;
        q.z = (R(1,0) - R(0,1)) * s;
    } else if(R(0,0) > R(1,1) && R(0,0) > R(2,2)) {
        // Cas x dominant
        double s = 2.0 * std::sqrt(1.0 + R(0,0) - R(1,1) - R(2,2));
        q.w = (R(2,1) - R(1,2)) / s;
        q.x = 0.25 * s;
        q.y = (R(0,1) + R(1,0)) / s;
        q.z = (R(0,2) + R(2,0)) / s;
    }
    // ... (branches y dominant et z dominant similaires)
    return q.Normalized();
}

```

4.5 TP — Semaine 4

TP 1 (1h30) — Quaternions complets

41. Vérifiez : $\text{Rotate}(\text{FromAxisAngle}(\{0,1,0\}, \pi/2), \{1,0,0\}) \approx \{0,0,-1\}$
42. Aller-retour Quat→Mat3→Quat sur 50 quaternions aléatoires (kEps près)
43. Vérifiez que $\text{Quat} \times \text{Quat.Inverse}() = \text{identité}$

TP 2 (1h30) — Animation SLERP

44. Animez une rotation entre deux orientations en 60 frames
45. Comparez visuellement SLERP vs LERP (LERP produit une vitesse angulaire non-uniforme)
46. Utilisez le rasteriseur logiciel du TP M03 pour afficher le cube en rotation

SEMAINE 5 — MODULE 05 : SVD — Décomposition en Valeurs Singulières

Durée : 6 heures sur 2 séances | Semestre 1 — NkMath | C++17 from scratch

5.1 Pourquoi la SVD est l'algorithme central du cours

La SVD est présente à chaque étape du pipeline AR : la calibration Zhang résout un système $Ax=0$ par SVD, la DLT pour l'homographie finit par une SVD, EPnP extrait la pose depuis la SVD d'une matrice 12×12 , la décomposition polaire $R=UV^T$ extrait la rotation pure. Ne pas comprendre la SVD, c'est travailler avec des boîtes noires incompréhensibles.

5.2 Signification géométrique de $A = U\Sigma V^T$

Toute matrice réelle $m \times n$ se décompose en $A = U\Sigma V^T$ où :

- U est $m \times m$ orthogonale — les vecteurs singuliers gauches (base orthonormale de l'espace image)
- Σ est $m \times n$ diagonale avec $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ — les valeurs singulières
- V est $n \times n$ orthogonale — les vecteurs singuliers droits (base orthonormale de l'espace d'entrée)

Géométriquement : A effectue 3 opérations successives : une rotation V^T dans l'espace d'entrée, un étirement Σ selon les axes canoniques, puis une rotation U dans l'espace de sortie.

Applications de la SVD dans le pipeline AR

Résolution de $Ax=0$: solution = dernière colonne de V (vecteur singulier minimal)

Pseudo-inverse A^+ : $A^+ = V\Sigma^+U^T$ (résout $Ax=b$ aux moindres carrés)

Calibration Zhang : extraction de K depuis les homographies

Homographie DLT : système $A(2n \times 9)x=0 \rightarrow$ solution SVD

EPnP pose estimation : système $A(2n \times 12)x=0 \rightarrow$ solution SVD

Décomposition polaire : $R = UV^T$ (rotation pure extraite d'une matrice quelconque)

5.3 Implémentation — structure SVDResult

```
// SVDResult : résultat de la décomposition  $A = U * \Sigma * V^T$ 
struct SVDResult {
    MatXd U;           //  $m \times m$  orthogonale
    VecXd sigma;       // valeurs singulières triées décroissantes
    MatXd V;           //  $n \times n$  orthogonale (attention :  $V$ , pas  $V^T$ )

    // Vérification :  $\|A - U*\Sigma*V^T\| < kEps$ 
    double residual(const MatXd& A) const;

    // Pseudo-inverse  $A^+ = V * \Sigma^+ * U^T$ 
    MatXd pseudoInverse(double threshold = 1e-10) const;

    // Solution de  $Ax=0$  : dernière colonne de  $V$ 
    VecXd nullSpaceVector() const;

    // Décomposition polaire  $A = R*S$  :  $R = U*V^T$ 
    MatXd rotationPart() const;
};

// Fonction principale
```

```
SVDResult svd(const MatXd& A);
```

5.4 Décomposition polaire et correction de signe

```
// Décomposition polaire :  $A = R * S$ 
//  $R = UV^T$  (rotation, matrice orthogonale)
//  $S = VEV^T$  (matrice symétrique positive)
// Application : forcer une matrice à être dans  $SO(3)$ 
Mat3d polarRotation(const Mat3d& A) {
    SVDResult svd = svd3x3(A);
    Mat3d R = svd.U * svd.V.Transposed();

    // Correction : si  $\det(R) < 0$ , R est une réflexion, pas une rotation
    // Inverser le signe de la dernière colonne de U pour forcer  $\det=+1$ 
    if( $\det(R) < 0$ ) {
        // Dernière colonne de U  $\times -1$ 
        for(int r=0; r<3; r++) svd.U(r,2) *= -1;
        R = svd.U * svd.V.Transposed();
    }
    return R;
}
```

5.5 TP — Semaine 5

TP 1 (1h30) — Tester l'implémentation SVD fournie

47. Testez SVD sur 20 matrices aléatoires : vérifiez $A = U\Sigma V^T$, $\|U\|=1$, $\|V\|=1$, σ triés
48. Testez sur une matrice rang-déficiente : les dernières σ doivent être ≈ 0
49. Vérifiez la pseudo-inverse sur un système 3×2 sur-déterminé

TP 2 (1h30) — Application : résolution d'homographie

50. Créez 4 correspondances $2D \rightarrow 2D$ (8 équations) sous forme $Ah=0$
51. Résolvez par SVD : h = dernière colonne de V
52. Recomposez H 3×3 et vérifiez les résidus de reprojection

SEMAINE 6 — MODULE 06 : NkImage — Traitement d'image from scratch

Durée : 9 heures sur 3 séances | Semestre 1 — NkMath | C++17 from scratch

6.1 Pourquoi NkImage est central dans le pipeline AR

Toute la chaîne de détection ArUco opère sur des images : la frame brute de la webcam, le résultat de la binarisation, la carte de contours, les patches de marqueurs recadrés. NkImage est le type de données central. Sans une implémentation correcte — avec le bon layout mémoire pour l'upload GPU, l'accès sub-pixel par interpolation bilinéaire, la convolution 2D, et le seuillage adaptatif via image intégrale — le pipeline de détection ne peut pas fonctionner.

6.2 Structure NkImage — layout mémoire RGBA

```
#pragma once
#include <cstdint>
#include <vector>
#include <cassert>
#include <cstring>
#include <string>

namespace NkMath {

class NkImage {
public:
    // Constructeurs
    NkImage() : m_width(0), m_height(0) {}
    NkImage(int w, int h) : m_width(w), m_height(h), m_data(w*h*4, 0) {}

    // Accesseur bounds-checked en debug
    uint8_t* At(int x, int y) {
        assert(x >= 0 && x < m_width && "x out of bounds");
        assert(y >= 0 && y < m_height && "y out of bounds");
        return m_data.data() + (y * m_width + x) * 4;
    }
    const uint8_t* At(int x, int y) const {
        assert(x >= 0 && x < m_width && y >= 0 && y < m_height);
        return m_data.data() + (y * m_width + x) * 4;
    }

    // Setters pratiques
    void SetPixelRGBA(int x, int y, uint8_t r, uint8_t g, uint8_t b, uint8_t
a=255) {
        uint8_t* p = At(x,y);
        p[0]=r; p[1]=g; p[2]=b; p[3]=a;
    }
    void SetPixelGray(int x, int y, uint8_t v) { SetPixelRGBA(x,y,v,v,v,255); }

    // Dimensions
    int Width() const { return m_width; }
    int Height() const { return m_height; }

    // Pointer direct vers les données (pour glTexImage2D)
    const uint8_t* Data() const { return m_data.data(); }
    uint8_t* Data() { return m_data.data(); }
```

```

size_t      DataSize() const { return m_data.size(); }

// Remplissage
void Fill(uint8_t r, uint8_t g, uint8_t b, uint8_t a=255);

// Sauvegarde PPM (format simple sans dépendance)
bool SavePPM(const std::string& path) const;
bool LoadPPM(const std::string& path);

// Conversion niveaux de gris (vers buffer séparé pour le pipeline AR)
std::vector<uint8_t> ToGrayscale() const;

private:
    int m_width, m_height;
    std::vector<uint8_t> m_data; // layout RGBA contigu
};

} // namespace NkMath

```

6.3 I/O sans dépendance — PPM et BMP

```

// Sauvegarder en PPM P6 (binaire) – le format le plus simple
bool NkImage::SavePPM(const std::string& path) const {
    FILE* f = fopen(path.c_str(), "wb");
    if(!f) return false;
    fprintf(f, "P6\n%d %d\n255\n", m_width, m_height);
    // PPM P6 = RGB sans alpha
    for(int y=0; y<m_height; y++)
        for(int x=0; x<m_width; x++) {
            const uint8_t* p = At(x,y);
            fwrite(p, 1, 3, f); // R, G, B seulement (pas A)
        }
    fclose(f);
    return true;
}

// Charger un PPM P6
bool NkImage::LoadPPM(const std::string& path) {
    FILE* f = fopen(path.c_str(), "rb");
    if(!f) return false;
    char magic[3];
    int w, h, maxVal;
    fscanf(f, "%s %d %d %d ", magic, &w, &h, &maxVal);
    if(magic[0]!='P' || magic[1]!='6') { fclose(f); return false; }
    *this = NkImage(w, h);
    for(int y=0; y<h; y++)
        for(int x=0; x<w; x++) {
            uint8_t rgb[3];
            fread(rgb, 1, 3, f);
            SetPixelRGBA(x, y, rgb[0], rgb[1], rgb[2], 255);
        }
    fclose(f);
    return true;
}

```

6.4 Interpolation bilinéaire


```

// Accès sub-pixel avec interpolation bilinéaire
// Retourne la couleur interpolée à la position flottante (fx, fy)
struct Color4 { double r, g, b, a; };

Color4 SampleBilinear(const NkImage& img, double fx, double fy) {
    // Coordonnées des 4 pixels voisins
    int x0 = (int)fx; int y0 = (int)fy;
    int x1 = std::min(x0+1, img.Width()-1);
    int y1 = std::min(y0+1, img.Height()-1);
    x0 = std::max(0, x0); y0 = std::max(0, y0);

    // Fractions sub-pixel
    double tx = fx - (int)fx; // [0, 1)
    double ty = fy - (int)fy;

    // Les 4 pixels voisins
    const uint8_t* p00 = img.At(x0, y0);
    const uint8_t* p10 = img.At(x1, y0);
    const uint8_t* p01 = img.At(x0, y1);
    const uint8_t* p11 = img.At(x1, y1);

    // Interpolation bilinéaire pour chaque canal
    Color4 c;
    for(int i=0; i<4; i++) {
        double v00 = p00[i], v10 = p10[i];
        double v01 = p01[i], v11 = p11[i];
        // Interpoler horizontalement, puis verticalement
        double top    = v00*(1-tx) + v10*tx;
        double bottom = v01*(1-tx) + v11*tx;
        (&c.r)[i] = top*(1-ty) + bottom*ty;
    }
    return c;
}

```

6.5 Image intégrale et seuillage adaptatif

```

// Image intégrale (Summed Area Table)
// Construction O(WH), requête rectangulaire O(1)
class IntegralImage {
public:
    explicit IntegralImage(const std::vector<uint8_t>& gray, int w, int h)
        : m_w(w), m_h(h), m_table(w*h, 0) {
        // Construire la table ligne par ligne
        for(int y=0; y<h; y++)
            for(int x=0; x<w; x++) {
                int idx = y*w + x;
                m_table[idx] = gray[idx]
                    + (x>0 ? m_table[idx-1] : 0)
                    + (y>0 ? m_table[idx-w] : 0)
                    - (x>0&&y>0 ? m_table[idx-w-1] : 0);
            }
    }

    // Somme du rectangle [x0,y0] → [x1,y1] en O(1)
    long long RectSum(int x0, int y0, int x1, int y1) const {
        x0 = std::max(0, x0); y0 = std::max(0, y0);
        x1 = std::min(m_w-1, x1); y1 = std::min(m_h-1, y1);
    }
};

```

```

        long long s = m_table[y1*m_w+x1];
        if(x0>0) s -= m_table[y1*m_w+(x0-1)];
        if(y0>0) s -= m_table[(y0-1)*m_w+x1];
        if(x0>0&& y0>0) s += m_table[(y0-1)*m_w+(x0-1)];
        return s;
    }

    int Area(int x0, int y0, int x1, int y1) const {
        return (x1-x0+1)*(y1-y0+1);
    }

private:
    int m_w, m_h;
    std::vector<long long> m_table;
};

// Seuillage adaptatif via image intégrale
NkImage AdaptiveThreshold(const NkImage& src, int blockSize=31, int offset=7) {
    auto gray = src.ToGrayscale();
    int w = src.Width(), h = src.Height();
    IntegralImage ii(gray, w, h);
    NkImage result(w, h);
    int half = blockSize / 2;
    for(int y=0; y<h; y++)
        for(int x=0; x<w; x++) {
            long long sum = ii.RectSum(x-half, y-half, x+half, y+half);
            int area = ii.Area(x-half, y-half, x+half, y+half);
            int mean = (int)(sum / area);
            uint8_t pix = gray[y*w+x];
            uint8_t out = (pix < mean - offset) ? 0 : 255;
            result.SetPixelGray(x, y, out);
        }
    return result;
}

```

6.6 TP — Semaine 6

TP 1 (1h30) — NkImage de base

53. Implémentez NkImage avec At, Fill, SetPixelRGBA, SavePPM, LoadPPM
54. Créez une image 512×512 avec un dégradé RGB et des formes géométriques (cercle, ligne, rectangle)
55. Sauvegardez en PPM et vérifiez visuellement

TP 2 (1h30) — SampleBilinear et Convolve

56. Implémentez SampleBilinear et testez avec des coordonnées fractionnaires
57. Implémentez Convolve générique avec kernel flottant de taille impaire
58. Appliquez flou Gauss 5×5 et Sobel horizontal/vertical sur une photo PPM
59. Comparez les résultats avant/après et mesurez le temps de traitement

TP 3 (1h30) — Image intégrale et seuillage

60. Implémentez IntegralImage et AdaptiveThreshold
61. Testez sur une photo d'un marqueur ArUco imprimé
62. Ajustez blockSize=31, offset=7 jusqu'à binarisation propre
63. Mesurez le temps de construction de l'image intégrale vs convolution naïve

